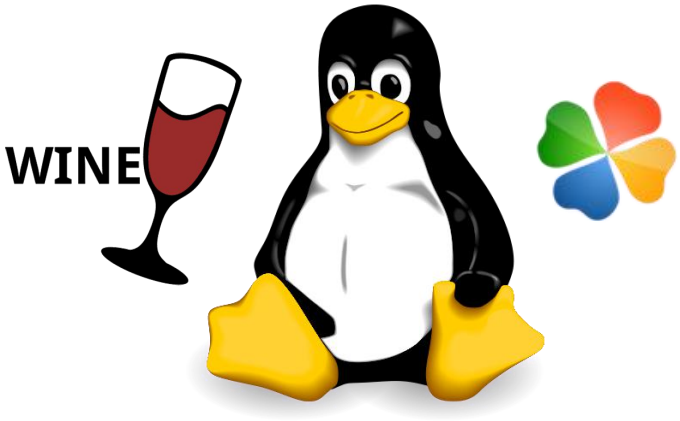
Portada

MANUAL BASICO PARA CREAR SCRIPTS EN PLAYONLINUX

Versión en español



Realizado por: Orlin Fabricio Lagos (GuerreroAzul)

Primera edición

La realizacion de este manual se realizo usando la herramienta de IA ChatGPT.
Enlace: https://chat.openai.com/

Se uso como base la documentacion de usuario de PlayOnLinux

Enlace: https://wiki.playonlinux.com/



Indice

Portada 1
Indice 2
Introducción 5
1. Conocimientos basicos 6
1.1. Que es bash? 6
1.2. Que es un script? 6
1.3. Requisitos minimos para la realización de un script. 6
1.4. Ejecutando el script 7
1.5. Firma del script 8
2. Funciones basicas 9
2.1. Comentarios 9
2.2. Mensajes 11
2.3. Cuadro de texto 13
2.4. Verdadero o Falso 14
2.5. Selección única 15
2.6. Seleccionando un archivo 16
2.7. Configuracion de imagenes 17
2.8. Presentación del programa 18
3. Variables 20

3.1. ¿Que es una variable? 20
3.2. Declaración de variables 20
3.3. Lectura de una variable 20
3.4. Almacenamiento de funciones `"$APP_ANSWER"` 22
3.5. Estandarización de variables 22
4. Estructuras 24
4.1. Estructura en secuencia. 24
4.2. Estructura de seleccion ó estructura condicional. 25
4.3. Estructura de Repetición(Bucles ó Ciclos). 27
- *Ciclo `for`* *27*
- *Ciclo `while`* *29*
- *Ciclo `do-while`* *31*

4.4. Estructura de Funciones (Procedimientos o Subrutinas). 32
4.5. Estandarización de codigo. 32
5. Condiciones 35
5.1. Que es una condicion en programacion? 35
6. Wine 36
6.1. ¿Que es wine? 36
6.2. Prefijos o Discos Virtuales 36
6.3. Ejecucion de un archivo 37
6.4. Mostrar mensaje de espera 37
6.5. Usar una version de wine especifica. 38
6.6. Usar una version de sitema operativo. 38
6.7. Estandarización de codigo. 38
7. Los archivos del sistema 40
7.1. Directorios temporales de PlayOnLinux. 40

7.2. Comandos de sistema bash. 41
- *Cambio del directorio 41*
- *Eliminar un archivo o directorio 41*
- *Copiar un archivo o directorio 41*
- *Mover o cambiar el nombre de un archivo o directorio 41*

8. Medios de instalación 42
8.1. Medio CD/DVD 42
8.2. Descargar un programa de internet 42
8.3. Seleccionar el metodo de instalacion 43
8.4. Creando un lanzador en POL 43
9. Mi primer script funcional 44
10. Traduccion de script 47
11. Final del manual 48



Introducción

Bienvenido al manual de uso de scripts en PlayOnLinux en español. PlayOnLinux (Lo llamaremos POL, para los amigos), es una interfaz gráfica para Wine, ofrece a los usuarios de Linux una solución eficaz para ejecutar aplicaciones y juegos de Windows en sus sistemas operativos nativos. Este manual está diseñado para ayudarte a aprovechar al máximo la potencia de los scripts en PlayOnLinux, facilitándote el proceso de instalación y configuración de tus aplicaciones favoritas.

Si bien PlayOnLinux se presenta comúnmente en inglés, esta guía busca brindarte la información esencial en español, para que puedas disfrutar de la versatilidad de esta herramienta sin barreras lingüísticas. A lo largo de estas páginas, exploraremos los conceptos básicos de los scripts, proporcionaremos instrucciones paso a paso y resolveremos posibles problemas que puedan surgir durante el proceso.

Ya seas un usuario principiante o experimentado, este manual tiene como objetivo servirte como una referencia completa para optimizar tu experiencia con PlayOnLinux y facilitar la integración de aplicaciones de Windows en tu entorno Linux. ¡Empecemos la aventura!



1. Conocimientos basicos

1.1. Que es bash?

NU Bash o simplemente Bash (Bourne-again shell) es un intérprete de comandos y un lenguaje de programacion en scripts utilizado principalmenteen **GNU/Linux** y **Mac OS X.**

1.2. Que es un script?

Un script es un programa o conjunto de instrucciones que se almacenan en un archivo de texto. Está compuesto por una serie de comandos que pueden ser ejecutados secuencialmente para realizar una tarea específica o una serie de tareas relacionadas.

PlayOnLinux tiene funciones avanzadas en Bash para facilitar el proceso de creación de scripts.

1.3. Requisitos minimos para la realización de un script.

Cada script de POL debe contener este código en cada uno de ellos:

```
#!/usr/bin/env playonlinux-bash
[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

POL_SetupWindow_Init

POL_SetupWindow_Close
exit 0
```

`#!/usr/bin/env`: Esta linea del codigo es estandar para todos los scritps shell, esta es llamada shebangs. Indica quee el interprete debe ser buscado en la ruta de retorno del sistema.

`playonlinux-bash`: Especifica el intérprete de Bash que debe ser utilizado. En el contexto de POL, este intérprete específico puede tener configuraciones o funciones adicionales adaptadas para trabajar en el entorno de POL.

`[ "$PLAYONLINUX" = "" ] && exit 0`: Verifica que la ejecucion se realice en en el entorno de POL, de lo contrario proceda a cerrarse.

`source "$PLAYONLINUX/lib/sources"`: Carga las librerias de POL el cual contiene los comandos y funciones que se usaran en el script.

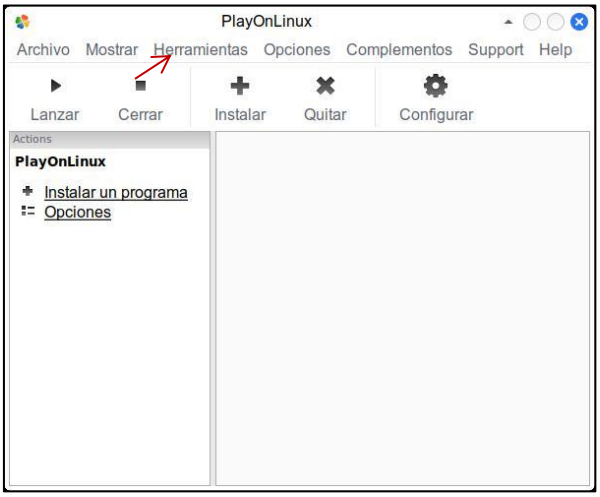
`POL_SetupWindow_Init`: Funcion que inicia la ventana de instalacion en POL.

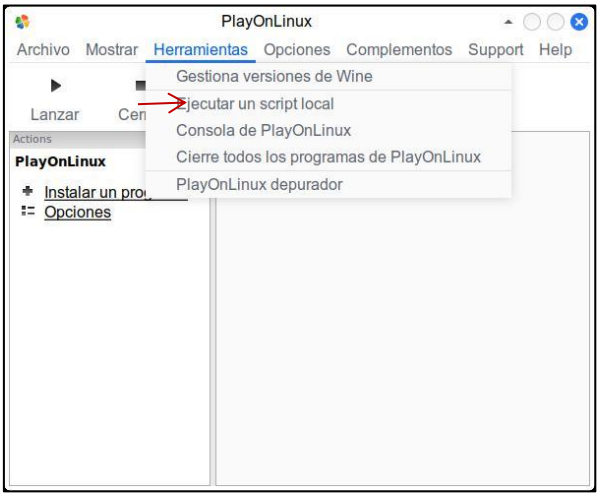
`POL_SetupWindow_Close`: Funcion que cierra la ventana de instalacion en POL.

`exit 0`: termina la ejecución del script. esta sera la ultima linea del script.

1.4. Ejecutando el script

Antes de ejecutar su script, es necesario que guarde su script en su editor de texto favorito con la extensión (**.sh**), luego use la función Ejecutar un script local desde el menú Herramientas de PlayOnLinux.

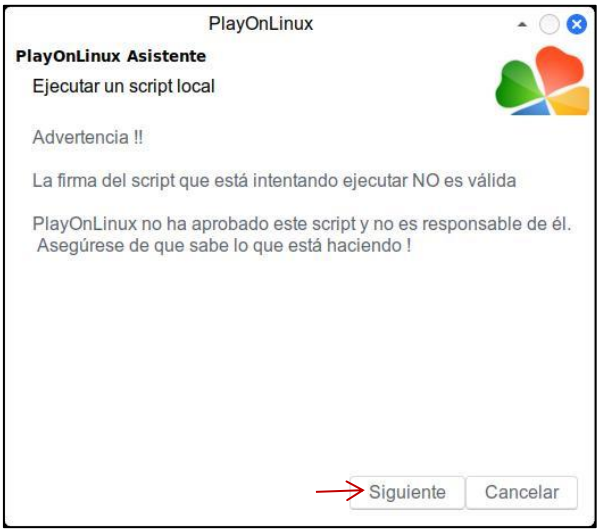


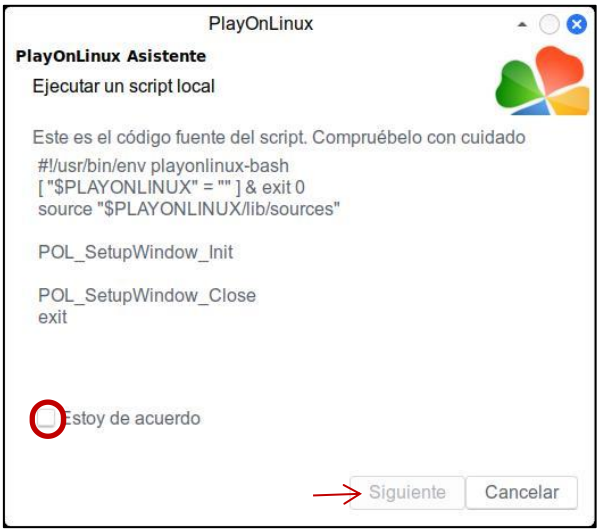


1.5. Firma del script

PlayOnLinux utiliza firma digital (autenticación) en los scripts para certificar que los scripts han sido validados por los programadores de PlayOnLinux. Todos los scripts disponibles desde la ventana de Instalación de PlayOnLinux están validados. Cada vez que intente ejecutar un script que no haya sido validado por los scripters de PlayOnLinux, recibirá un mensaje de advertencia que le informará que el script no tiene una firma válida.

Debe hacer clic en el botón Siguiente para que PlayOnLinux le muestre la fuente del script, luego hacer clic en la casilla Acepto, antes de que pueda (finalmente) ejecutar el script.







2. Funciones basicas

En este capítulo introducimos las funciones basicas y presentamos varios ejemplos que ilustran muchas características importantes del script POL.

2.1. Comentarios

Los comentarios en programación son textos descriptivos que se incluyen en el código fuente para proporcionar información adicional sobre el propósito, funcionamiento o cualquier otro aspecto relevante del código. Estos comentarios son ignorados por el compilador o intérprete y no afectan la ejecución del programa. Aquí hay algunas razones clave por las que se utilizan comentarios:

- **Explicación del Código**: Los comentarios ayudan a los programadores (incluido el propio autor del código en el futuro) a entender cómo funciona un fragmento de código. Pueden proporcionar aclaraciones sobre la lógica detrás de una sección de código, explicar decisiones de diseño o destacar detalles importantes.
- **Documentación del Código**: Los comentarios también pueden servir como documentación para el código, especialmente en funciones, clases o módulos más grandes. La documentación describe la funcionalidad, los parámetros de entrada y salida, y cualquier otro detalle relevante para el uso adecuado del código.
- **Marcadores y Recordatorios**: Los comentarios pueden actuar como marcadores o recordatorios para el programador. Pueden indicar tareas pendientes, problemas conocidos o áreas que necesitan atención en el futuro.

- **Desactivar Temporalmente Código**: Al agregar comentarios alrededor de un bloque de código, se puede "desactivar" temporalmente ese código sin tener que eliminarlo. Esto puede ser útil durante la depuración o al probar diferentes secciones del código.
- **Colaboración**: Los comentarios también facilitan la colaboración entre programadores. Cuando varios desarrolladores trabajan en un proyecto, los comentarios proporcionan información crucial sobre la intención detrás de ciertas decisiones de implementación.

Es importante escribir comentarios claros y significativos para que otras personas (o incluso tú mismo en el futuro) puedan entender y mantener el código de manera efectiva. Un buen equilibrio entre comentarios informativos y un código claro puede hacer que el mantenimiento y la comprensión del código sean mucho más sencillos.

Para utilizar un comentario en los scripts con un `#` al principio de comentario, todo lo que este despues del `#` sera tomado como comentario, tambien se usa `:''` para segmentos con varias lineas.

Los comentarios en un script se usa tambien de seccion informativa, el cual este incluye datos importantes sobre el script. Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing
[GuerrreroAzul] (2024-02-01 16:00 GMT-6) Wine 8.0.2 / Ubuntu 22.04.3 LTS
  Script creation

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'
```

```
[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init

# GuerreroAzul: This is my personal comment.

# End script
POL_SetupWindow_Close
exit 0
```

Los campos `Date`(Fecha), `Last revision`(Ultima revisión), `Wine version used`(Version de wine usada) y `Distribution used to test`(Distribucion usada para pruebas). Esta informacion esta redirigida a `CHANGELOG`(Registro de cambios), en este apartado se lleva los registros de los cambios, debera llevar el nombre del usuario, Fecha del cambio(Se puede incluir la zona horaria), version de wine y la version del sistema en donde se estan realizando las pruebas. En la segunda linea se da el detalles de los cambios realizados al script.

En `REFERENCE`(Referencias), se colocan los enlaces de apoyo que se uso para el script, este lleva el nombre del usuario, una breve descripcion del enlace y el enlace.

Al momento de comentar algo, es preferible que al principio del comentario colocar el nombre del usuario que comenta, para llevar un registro mas limpio.

2.2. Mensajes

Para mostrar una ventana con un texto usamos el siguiente comando:

```
POL_SetupWindow_message "Mensaje" "Titulo"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail
```

```
CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init

POL_SetupWindow_message "¡Hola! \nBienvenido." "PlayOnLinux"

# End script
POL_SetupWindow_Close
exit 0
```



Observe que los caracteres `"\n"` no aparecieron en pantalla. La diagonal invertida (\) se conoce como *carácter de escape*. Éste indica que se espera que en el texto haga algo fuera de lo ordinario. Cuando una diagonal invertida se encuentra dentro de una cadena, el compilador ve el siguiente carácter y lo combina con la diagonal invertida para formar una secuencia de escape. La secuencia de escape `"\n"` significa *nueva línea*.

Aqui muestro una lista de las secuencias de escape para POL:

- `"\n"`: Nueva linea.
- `"\t"`: Tabulacion horizontal.
- `"\\"`: Barra invertida.
- `"\'"`: Comilla simple.
- `"\""`: Comilla doble.

2.3. Cuadro de texto

¿Quieres hacerle una pregunta al usuario? La siguiente función fue creada para este propósito. Agregue el siguiente código a su script:

```
POL_SetupWindow_textbox "Mensaje" "Titulo"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init

POL_SetupWindow_textbox "¡Hola! \n¿Cual es tu nombre?" "PlayOnLinux"

# End script
POL_SetupWindow_Close
exit 0
```



Los datos generados seran almacenados en la variable `"$APP_ANSWER"`, el cual esto sera visto con mas detalle en el Capitulo 3.

2.4. Verdadero o Falso

¿Quieres que el usuario pueda responder sí o no? Entonces usa esta función:

```
POL_SetupWindow_question "Pregunta" "Titulo"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing

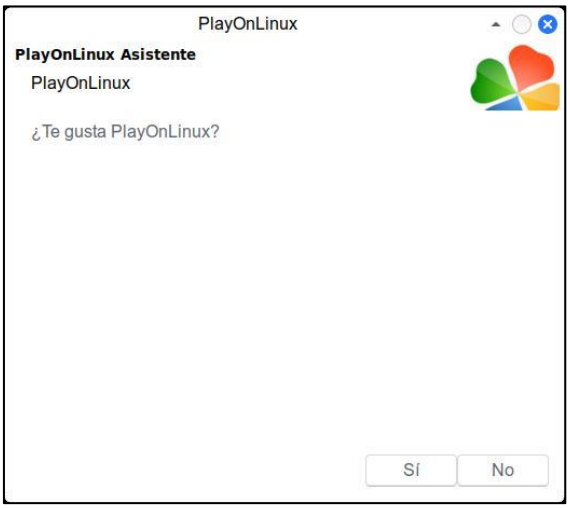
REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init
```

```
POL_SetupWindow_question "¿Te gusta PlayOnLinux?" "PlayOnLinux"

# End script
POL_SetupWindow_Close
exit 0
```



Los datos generados seran almacenados en la variable `"$APP_ANSWER"`, el cual contreda los valores `"TRUE"`, en caso de ser si o `"FALSE"`, en caso de ser no. Esto sera visto con mas detalle en el Capitulo 3.

2.5. Selección única

Al igual que con las dos funciones anteriores, puedes crear un menú para POL:

```
POL_SetupWindow_menu "Mensaje" "Titulo" "Lista del menu" "El separador"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
```

```
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init

POL_SetupWindow_menu "Select the language of your preference:\n"
"PlayOnLinux" "English, Spanish" ", "

# End script
POL_SetupWindow_Close
exit 0
```



Un separador es el espacio que se encuentra entre varios texto. en nuestro ejemplo usamos el separador `", "`(coma y espacio) para marca la lista a generar.

Los datos generados seran almacenados en la variable `"$APP_ANSWER"`, el cual contreda los valores `"English"` o `"Spanish"`. Esto sera visto con mas detalle en el Capitulo **3. Variables.**

2.6. Seleccionando un archivo

Puede pedirle al usuario que seleccione un archivo de su computadora de la siguiente manera:

```
POL_SetupWindow_browse "Mensaje" "Titulo"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
```

```
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 9.0 / Linux Mint 21.3 x86_64
  Testing

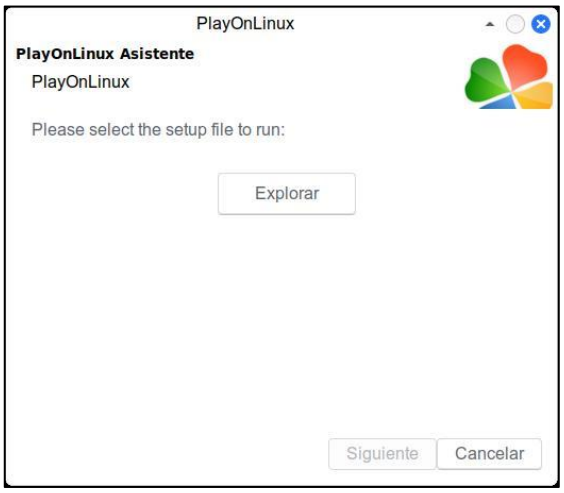
REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

# Starting the script
POL_SetupWindow_Init

POL_SetupWindow_browse "$(eval_gettext 'Please select the setup file to
 run.')" "PlayOnLinux"

# End script
POL_SetupWindow_Close
exit 0
```



La función "eval_gettext" en scripts POL se utiliza para traducir cadenas de texto a diferentes idiomas. En el contexto de PlayOnLinux, esto facilita la internacionalización de los scripts, permitiendo que los mensajes y las interfaces de usuario se presenten en el idioma preferido del usuario.

2.7. Configuracion de imagenes

Este comando tiene un uso estetico el cual permite personalizar la imagen del script, dando asi una visualizacion mas estetica ha este:

```
POL_GetSetupImages "Icono" "Banner" "Editor" "Titulo"
```

2.8. Presentación del programa

Este comando es muy importante. Le permite presentar el programa al usuario antes de que se inicie la instalación:

```
POL_SetupWindow_presentation "Nombre del programa" "Editor de el programa"
"sitio del editor" "Tu Nick" "Prefijo del programa (Ver Cap. 5)"
```

Ejemplo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setup Image
POL_GetSetupImages "https://i.imgur.com/licFVuF.png"
"https://i.imgur.com/ff6PkEZ.png" "Microsoft Office 2016"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "Microsoft" "Microsoft"
"http://www.microsoft.com"
"Quentin Pâris, Eduardo Lucio and N0rbert, GuerreroAzul, Dingo35"
"Microsoft Office 2016"
```

```
# End script
POL_SetupWindow_Close
exit 0
```



Dato a lo aprendido. se usara este script como base para los script de los siguientes capitulo, lo cual se debera tomar como estandar para los codigos que se usaran.



3. Variables

3.1. ¿Que es una variable?

Una variable es un contenedor para almacenar datos que pueden ser modificados y referenciados en un programa o script. Las variables permiten a los programadores almacenar información, realizar cálculos y manipular datos de manera dinámica durante la ejecución del programa..

3.2. Declaración de variables

Para realizar la declaración de una variable se realiza de la siguiente manera:

```
Texto="Hola, mundo."
Numero=42
Decimal=3.14
Arreglo=("Cosa 1" "Cosa 2" "Cosa 3")
Booleano="TRUE"
Ruta="/ruta/del/archivo.txt"
```

3.3. Lectura de una variable

Para realizar la lectura de una variable es necesario usar el símbolo "$" al principio de nombre de la variable:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail
```

```
CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

Fruta="Melon"

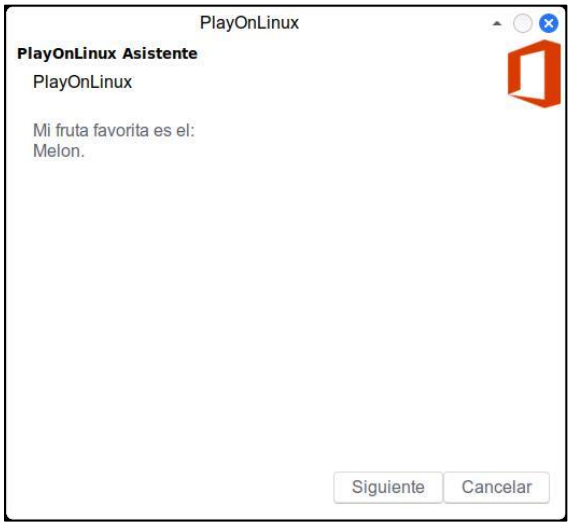
#Setup Image
POL_GetSetupImages "https://i.imgur.com/licFVuF.png"
"https://i.imgur.com/ff6PkEZ.png"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "Microsoft" "Microsoft"
"http://www.microsoft.com"
"Quentin Pâris, Eduardo Lucio and N0rbert, GuerreroAzul, Dingo35"
"Microsoft Office 2016"

POL_SetupWindow_message "Mi fruta favorita es el: \n$Fruta."

# End script
POL_SetupWindow_Close
exit 0
```



3.4. Almacenamiento de funciones "$APP_ANSWER"

todas las funciones que contienen una acción con el usuario son almacenadas en la "$APP_ANSWER" esta variable es la que usaremos para detectar la selección del usuario.

Para "POL_SetupWindow_browse", es la ruta al archivo seleccionado por el usuario que se almacena en "$APP_ANSWER".

Para "POL_SetupWindow_question", "$APP_ANSWER" contendrá "TRUE" si el usuario respondió que sí, y "FALSE" si respondió que no.

3.5. Estandarización de variables

Hay variables que se usan para realizar la configuracion estandar del script, los nombres de estan variables deberan de uso exclusivo del script, esto con el fin de mantener una codigo limpio y organizado. Aqui muestro el estandar que se estará usando para los demas codigos en el siguiente capitulo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
```

```
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

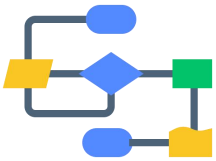
#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# End script
POL_SetupWindow_Close
exit 0
```

- TITLE: Titulo del script
- PREFIX: Nombre de la unidad virtual (Capitulo 6)
- WINEVERSION: Version de Wine (Capitulo 5)
- OSVERSION: Version del sistema virtual (Capitulo 6)
- EDITHOR: Autores del script
- COMPANY: Nombre de la compañia
- HOMEPAGE: Pagina web de la compañia
- LOGO: Logotipo del programa (64x64 pixeles)
- BANNER: Banner del programa (150x356 pixeles)



4. Estructuras

las estructuras se refieren a formas organizadas de organizar y controlar el flujo de ejecución de un programa. Estas estructuras proporcionan un marco para la escritura de código que permite realizar tareas específicas y controlar la lógica del programa. Las estructuras fundamentales incluyen:

4.1. Estructura en secuencia.

Es la estructura más simple y directa. Las instrucciones se ejecutan en orden, una después de la otra.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
```

```
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# Script start
POL_SetupWindow_textbox "¡Hola! \n¿Cual es tu nombre?" "PlayOnLinux"
POL_SetupWindow_message "¡Hola! \nBienvenido $APP_ANSWER." "PlayOnLinux"

# End script
POL_SetupWindow_Close
exit 0
```





4.2. Estructura de seleccion ó estructura condicional.

Permite que el programa tome decisiones basadas en condiciones. Las estructuras típicas son `if`, `elif`, `else` y `fi`.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# Script start
POL_SetupWindow_menu "¿Qué cenaremos hoy?:\n"
 "PlayOnLinux" "Zanahorias, Papas, Cebollas" ", "

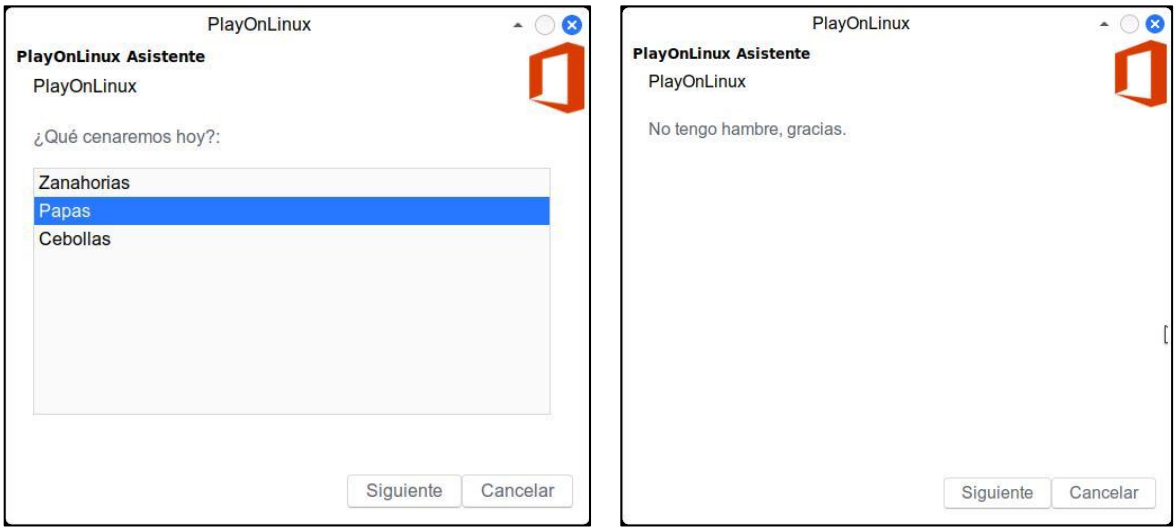
if [ "$APP_ANSWER" = "Zanahorias" ]; then
    POL_SetupWindow_message "Excelente para la vista." "PlayOnLinux"
elif [ "$APP_ANSWER" = "Papas" ]; then
    POL_SetupWindow_message "Es mi platillo favorito." "PlayOnLinux"
 else
    POL_SetupWindow_message "No tengo hambre, gracias." "PlayOnLinux"
```






```
fi

# End script
POL_SetupWindow_Close
exit 0
```



4.3. Estructura de Repetición(Bucles ó Ciclos).

Permite que un conjunto de instrucciones se repita múltiples veces hasta que se cumple una condición específica. Los bucles típicos son `for`, `while`, y `do-while`.

❖ **Ciclo `for`**

Un ciclo `for` es una estructura de control de flujo en programación que permite ejecutar un bloque de código repetidamente para un rango específico de valores o elementos de una lista. La idea principal es iterar sobre una secuencia de elementos y realizar alguna operación en cada iteración. El ciclo `for` simplifica la escritura y ejecución de operaciones repetitivas al automatizar el proceso de iteración.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
```

```
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

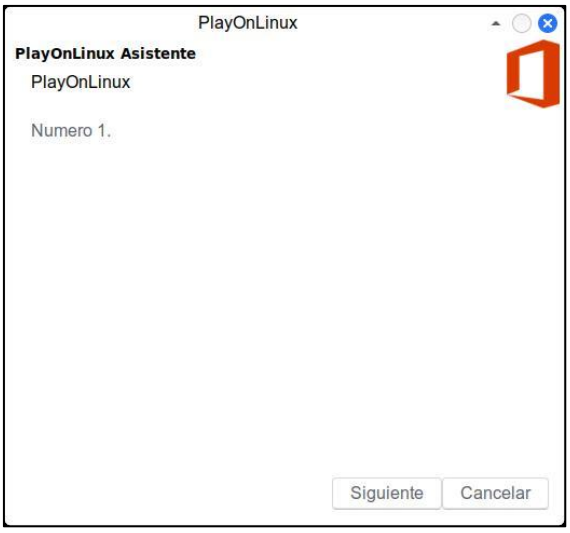
#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

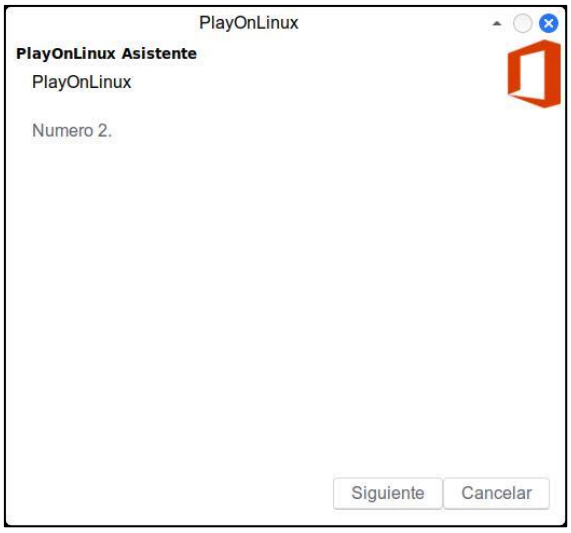
# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# Script start
for i in {1..2}; do
    POL_SetupWindow_message "Numero $i." "PlayOnLinux"
done

# End script
POL_SetupWindow_Close
exit 0
```





❖ Ciclo `while`

Un ciclo `while` es una estructura de control de flujo en programación que permite ejecutar repetidamente un bloque de código mientras una condición específica sea verdadera. En otras palabras, el bloque de código dentro del ciclo `while` se seguirá ejecutando mientras la condición sea evaluada como verdadera.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
```

```
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"
Contador=1

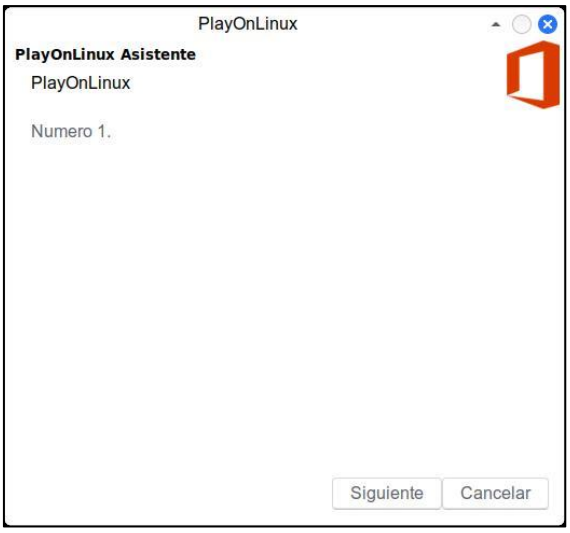
#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

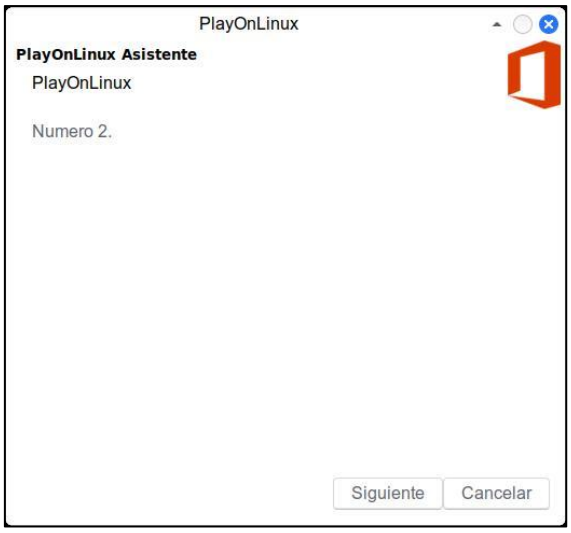
# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# Script start
while [ $Contador -le 2 ]; do
    POL_SetupWindow_message "Numero $Contador." "PlayOnLinux"
    ((Contador++))
done

# End script
POL_SetupWindow_Close
exit 0
```





❖ Ciclo `do-while`

La estructura de control de flujo `do-while` es un tipo de bucle en programación que ejecuta un bloque de código al menos una vez, y luego continúa repitiendo ese bloque mientras una condición específica sea verdadera. La característica distintiva del bucle `do-while` es que la condición se evalúa después de que el bloque de código se ha ejecutado al menos una vez.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"
Contador=1

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

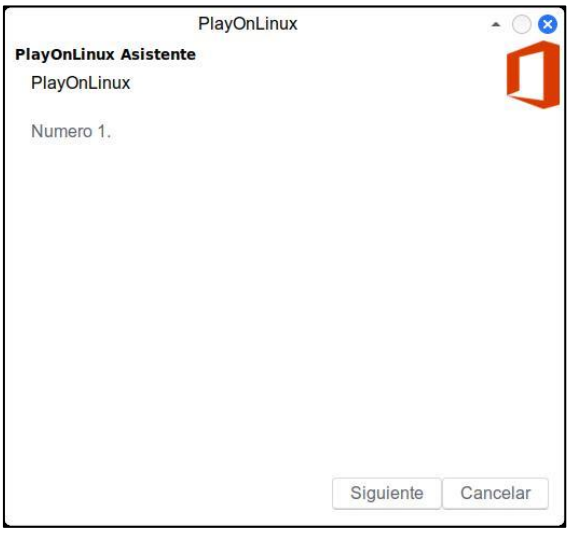
# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
```

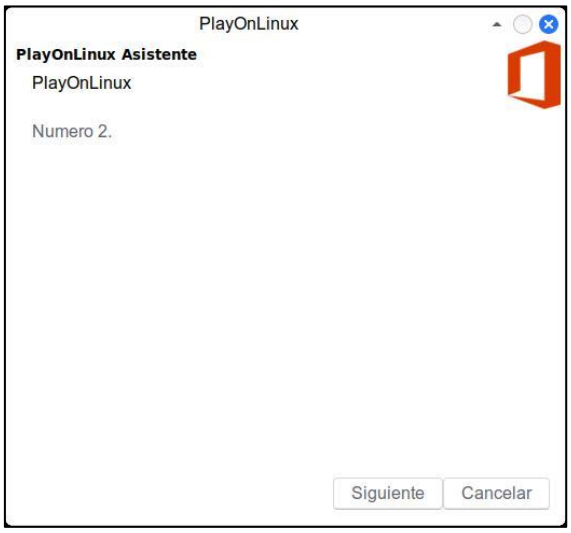
```
"$TITLE"

# Script start
while true; do
    POL_SetupWindow_message "Numero $Contador." "PlayOnLinux"
    ((Contador++))

    if [ $Contador -gt 2 ]; then
       break
    fi
done

# End script
POL_SetupWindow_Close
exit 0
```





4.4. Estructura de Funciones (Procedimientos o Subrutinas).

Agrupan un conjunto de instrucciones bajo un nombre, permitiendo que se llamen y ejecuten en cualquier parte del programa. Este se vera en mayor profundidad en https://wiki.playonlinux.com/index.php/Scripting_-_Chapter_11:_List_of_Functions

4.5. Estandarización de codigo.

Ya que conocemos las estructuras de programacion podemos avanzar con unas comprobaciones que se tienen que realizar antes de proseguir con el siguiente capitulo, de ahora en adelante se usará el siguiente codigo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

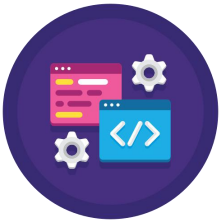
# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# PlayOnLinux Version Check
POL_RequiredVersion 4.3.4 || POL_Debug_Fatal "$TITLE won't work with
$APPLICATION_TITLE $VERSION\nPlease update!"

# Check winbind library is installed.
if [ "$POL_OS" = "Linux" ]; then
  wbinfo -V || POL_Debug_Fatal "$(eval_gettext 'Please install winbind
  before installing.')" "$TITLE!"
fi
```

```
# Script start

# End script
POL_SetupWindow_Close
exit 0
```



5. Condiciones

5.1. Que es una condicion en programacion?

las condiciones son expresiones que se evalúan como verdaderas o falsas, y se utilizan para controlar el flujo de ejecución del programa. Permiten tomar decisiones basadas en ciertos estados o valores, lo que hace que el código sea más dinámico y adaptable a diferentes situaciones.

- **== ó = (-eq)**: Verdadero si los operadores son iguales.
- **!= (-ne)**: Verdadero si los operadores no son iguales.
- **> (-ne)**: Verdadero si el operador de la izquierda es mayor.
- **>= (-ge)**: Verdadero si el operador de la izquierda es mayor o igual.
- **< (-lt)**: Verdadero si el operador de la izquierda es menor.
- **<= (-le)**: Verdadero si el operador de la izquierda es menor o igual.

Estos operadores se utilizan dentro de corchetes [ ] o dobles corchetes [[ ]] para formar expresiones condicionales.



6. Wine

6.1. ¿Que es wine?

Wine es una implementación gratuita de la API de Windows; es decir, permite utilizar Linux o Mac OS X para ejecutar software diseñado para Microsoft Windows. Está desarrollado por WineHQ. Este es el motor detrás de PlayOnLinux/PlayOnMac.

En este capítulo, le mostraré algunas funciones importantes de PlayOnLinux con respecto a Wine, pero no todas. Para las funciones más técnicas, debes leer la Lista de funciones del capitulo 12.

6.2. Prefijos o Discos Virtuales

Un "prefijo" se refiere a un entorno de Wine independiente que se utiliza para instalar y ejecutar aplicaciones de Windows de manera aislada en un sistema Linux. Los prefijos en POL son una característica clave que permite gestionar y ejecutar múltiples aplicaciones de Windows de forma separada, con diferentes configuraciones y dependencias de bibliotecas.

Cada prefijo en POL tiene su propio directorio en el sistema de archivos que contiene una instalación separada de Wine, así como sus propios ajustes de configuración. Esto significa que puedes tener diferentes versiones de Wine, bibliotecas y configuraciones para diferentes aplicaciones, lo que ayuda a evitar conflictos entre aplicaciones y a mantener la estabilidad del sistema.

```
POL_Wine_SelectPrefix "Nombre de la unidad virtual"
```

6.3. Ejecucion de un archivo

Se utiliza la función `POL_Wine` se utiliza para ejecutar archivos ejecutables de Windows (.exe) dentro de un prefijo de Wine específico. Esta función se encarga de configurar el entorno de Wine apropiado antes de ejecutar el archivo. Aquí hay algunos tipos de archivos que puedes ejecutar con la función `POL_Wine`:

- **Archivos ejecutables de Windows (.exe)**: Puedes ejecutar aplicaciones de Windows directamente desde un archivo ejecutable .exe utilizando la función `POL_Wine`. Esto incluye aplicaciones de instalación, programas portátiles y juegos.
- **Archivos de instalación de Windows (.msi)**: Si tienes un archivo de instalación de Windows en formato MSI (Microsoft Installer), puedes utilizar la función `POL_Wine` para ejecutarlo y realizar la instalación como lo harías en Windows.
- **Archivos de instalación de otros formatos**: Aunque menos comunes, algunos programas de Windows pueden utilizar formatos de instalación diferentes, como archivos de instalación **.msu**, **.bat**, **.cmd**, entre otros. Siempre que el archivo contenga instrucciones para instalar o ejecutar un programa en Windows, es posible que puedas ejecutarlo con `POL_Wine`.

```
POL_Wine "ruta/al/archivo.exe"
```

6.4. Mostrar mensaje de espera

A veces resulta útil mostrar un mensaje de "espere" mientras se ejecutan los comandos. Esto es particularmente útil para el comando POL_Wine que durará mientras el programa que inició no se haya completado (por lo tanto, en el caso de una instalación, hasta que finalice la instalación).

```
POL_SetupWindows_wait "Mensaje" "Titulo"
```

6.5. Usar una version de wine especifica.

Es posible asignar alguna versión de Wine a un juego (independientemente de la versión de Wine instalada en todo el sistema). Esto es particularmente útil para juegos o aplicaciones grandes, cuyo rendimiento puede variar de una versión a otra de Wine. Esto también se puede utilizar si algún software dejó de funcionar con versiones recientes de Wine.

```
POL_Wine_PrefixCreate "Version de wine"
```

6.6. Usar una version de sitema operativo.

```
Set_OS "Versión de sitema operativo de wine"
```

6.7. Estandarización de codigo.

Usando los comando de este capitulo usaremos los siguientes para establecer las estandarizacion correspondiente, lo cual de ahora en adelante se usará el siguiente codigo:

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-02 09:30 GMT-6) Wine 8.0.2 / Linux Mint 21.3 x86_64
  Testing

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Microsoft Office 2016"
```

```
PREFIX="Office2016"
WINEVERSION="8.0.2"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="Microsoft®"
HOMEPAGE="https://Microsoft.com"
LOGO="https://i.imgur.com/licFVuF.png"
BANNER="https://i.imgur.com/ff6PkEZ.png"

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# PlayOnLinux Version Check
POL_RequiredVersion 4.3.4 || POL_Debug_Fatal "$(eval_gettext 'TITLE won\'t
work with $APPLICATION_TITLE $VERSION\nPlease update!')"

# Check winbind library is installed.
if [ "$POL_OS" = "Linux" ]; then
  wbinfo -V || POL_Debug_Fatal "$(eval_gettext 'Please install winbind
  before installing.')" "$TITLE!"
fi

# Prepare resources for installation!
POL_SetupWindow_browse "$(eval_gettext 'Please select the setup file to
run.')" "$TITLE"
Setup="$APP_ANSWER"

POL_Wine_SelectPrefix "$PREFIX"
POL_Wine_PrefixCreate "$WINEVERSION"
Set_OS "$OSVERSION"

# Script start
POL_SetupWindows_wait "$(eval_gettext 'Please wait.\nInstallation in
progress.')" "$TITLE"
POL_Wine "$Setup"

# End script
POL_SetupWindow_Close
exit 0
```



7. Los archivos del sistema

Hay un par de buenas formas de manipular sus archivos y directorios en Bash. Le daremos los comandos más importantes para escribir scripts de PlayOnLinux.

7.1. Directorios temporales de PlayOnLinux.

Puede utilizar un directorio temporal para su script cuando se ejecute. Para hacer eso:

```
POL_System_TmpCreate "Nombre del directorio temporal"
```

Creará una variable `$POL_System_TmpDir` que contendrá la ruta del directorio temporal. En este directorio, podrá almacenar y modificar todos los archivos que desee, durante toda la duración del script.

Se recomienda que los scripts de instalación verifiquen primero (en la medida de lo posible) que todos los recursos necesarios estén disponibles antes de la creación del prefijo y la instalación en sí. `POL_System_TmpCreate` puede resultar muy útil para este fin, porque no requiere que exista ningún prefijo.

Para eliminar el directorio temporal:

```
POL_System_TmpDelete
```

7.2. Comandos de sistema bash.

❖ Cambio del directorio

```
cd "$HOME" #Directorio Personal
```

❖ Eliminar un archivo o directorio

Este comando is muy peligroso, lo cual no se recomienda ser usado.

```
rm "Directorio o archivo a eliminar"
```

❖ Copiar un archivo o directorio

```
cp "Directorio o archivo a copia"
cp -r "/home/user/My Folder" "/home/user/Desktop/Test/"
```

❖ Mover o cambiar el nombre de un archivo o directorio

```
mv "/home/user/file.txt" "home/user/data.txt"
mv "/home/user/file.txt" "/home/user/Desktop/file.txt"
mv "/home/user/file.txt" "/home/user/Desktop/data.txt"
```



8. Medios de instalación

8.1. Medio CD/DVD

PlayOnLinux permite instalar fácilmente un software desde un CD/DVD.

```
POL_SetupWindow_cdrom
```

La ruta al CD/DVD se almacenará en la variable $CDROM (por ejemplo, el valor de $CDROM podría ser: /media/cdrom0).

8.2. Descargar un programa de internet

Con PlayOnLinux puedes descargar archivos fácilmente desde Internet. Esto es especialmente útil para el software que está disponible de forma gratuita en la red.

```
POL_Download "URL" "Resumen MD5"
```

El resumen MD5 es un código que se parece a b1bbd74395a34ff7fd069d3b6fe23016, que es único para cada archivo. Se utiliza para asegurarse de que el archivo descargado no esté dañado (como sucede a veces debido a errores de descarga). Para calcular el resumen MD5 de un archivo, use el siguiente comando en una consola/terminal:

```
md5sum "Ruta del archivo"
```

El archivo descargado se escribirá en el directorio de trabajo actual (seleccionado con el comando cd). La mayoría de las veces, el lugar correcto para los archivos descargados es el directorio temporal, creado con el comando

`POL_System_TmpCreate`. Para colocar archivos en el directorio temporal, use el siguiente comando justo antes de POL_Download :

```
cd "$POL_System_TmpDir"
```

8.3. Seleccionar el metodo de instalacion

¿Cómo le pregunto al usuario qué método de instalación quiere utilizar? ¡Poof! Aquí viene la función `POL_SetupWindow_InstallMethod`. Esta función le preguntará al usuario qué tipo de medio desea utilizar para instalar un programa.

```
POL_SetupWindow_InstallMethod "Metodo de instalación"
```

Existen muchos métodos de instalación diferentes, pero comencemos con los 4 más simples (y los más comunes):

- **LOCAL**: El usuario seleccionará un archivo de su computadora.
- **DOWNLOAD**: Descargue el archivo de instalación de Internet.
- **CD**: Instala el programa desde un CD.
- **DVD**: Instala el programa desde un DVD.

Los métodos disponibles para el usuario deben estar separados por comas. El método seleccionado por el usuario será devuelto en la variable $INSTALL_METHOD . Luego puede usar una combinación de if , then , elif y fi (declaraciones condicionales de Bash) para determinar qué método se ha elegido y actuar en consecuencia. Los detalles se darán en el próximo capítulo.

8.4. Creando un lanzador en POL

Crear un lanzador (o acceso directo) en PlayOnLinux es sumamente fácil, mira:

```
POL_Shortcut "archivo.exe" "Nombre del acceso directo"
```

No debes especificar la ruta completa al ejecutable, PlayOnLinux la encontrará automáticamente.



9. Mi primer script funcional

Con todo lo aprendido en los anteriores capitulo ya estamos listos para realizar nuestro primer script. este debera llevar los puntos de estandarizacion que se han marcado en todo el manual.

```
#!/usr/bin/env playonlinux-bash
: '
Date: See changelog.
Last revision: See changelog.
Wine version used: See changelog.
Distribution used to test: See changelog.
Author: GuerreroAzul
License: Retail

CHANGELOG
[GuerrreroAzul] (2024-02-06 14:00 GMT-6) Wine 8.6 / Linux Mint 21.3 x86_64
  Creation of the script

REFERENCE
GuerreroAzul: Documentation POL. - https://wiki.playonlinux.com/
'

[ "$PLAYONLINUX" = "" ] && exit 0
source "$PLAYONLINUX/lib/sources"

#Setting
TITLE="Winrar"
PREFIX="winrar"
CATEGORY="Accesories;"
WINEVERSION="8.6"
OSVERSION="win7"
EDITHOR="GuerreroAzul"
COMPANY="LABRAR®"
HOMEPAGE="https://www.win-rar.com/"
LOGO="https://i.imgur.com/fKDpsqH.png"
BANNER="https://imgur.com/2sJkq3I.png"
DOWNLOAD_URL="https://www.win-rar.com/fileadmin/winrar-versions/winrar/
winrar-x32-624.exe"
```

```
FILE_INSTALL="winrar-x32-624.exe"
MD5_CHECKSUM="e664a3f2485587e559835e19238e0d3"

#Setup Image
POL_GetSetupImages "$LOGO" "$BANNER" "$TITLE"

# Starting the script
POL_SetupWindow_Init

# Welcome message
POL_SetupWindow_presentation "$TITLE" "$COMPANY" "$HOMEPAGE" "$EDITHOR"
"$TITLE"

# PlayOnLinux Version Check
POL_RequiredVersion 4.3.4 || POL_Debug_Fatal "$(eval_gettext 'TITLE won\'t
work with $APPLICATION_TITLE $VERSION\nPlease update!')"

# Check winbind library is installed.
if [ "$POL_OS" = "Linux" ]; then
  wbinfo -V || POL_Debug_Fatal "$(eval_gettext 'Please install winbind
  before installing.')" "$TITLE!"
fi

# Prepare resources for installation!
POL_Wine_SelectPrefix "$PREFIX"
POL_Wine_PrefixCreate "$WINEVERSION"
Set_OS "$OSVERSION"

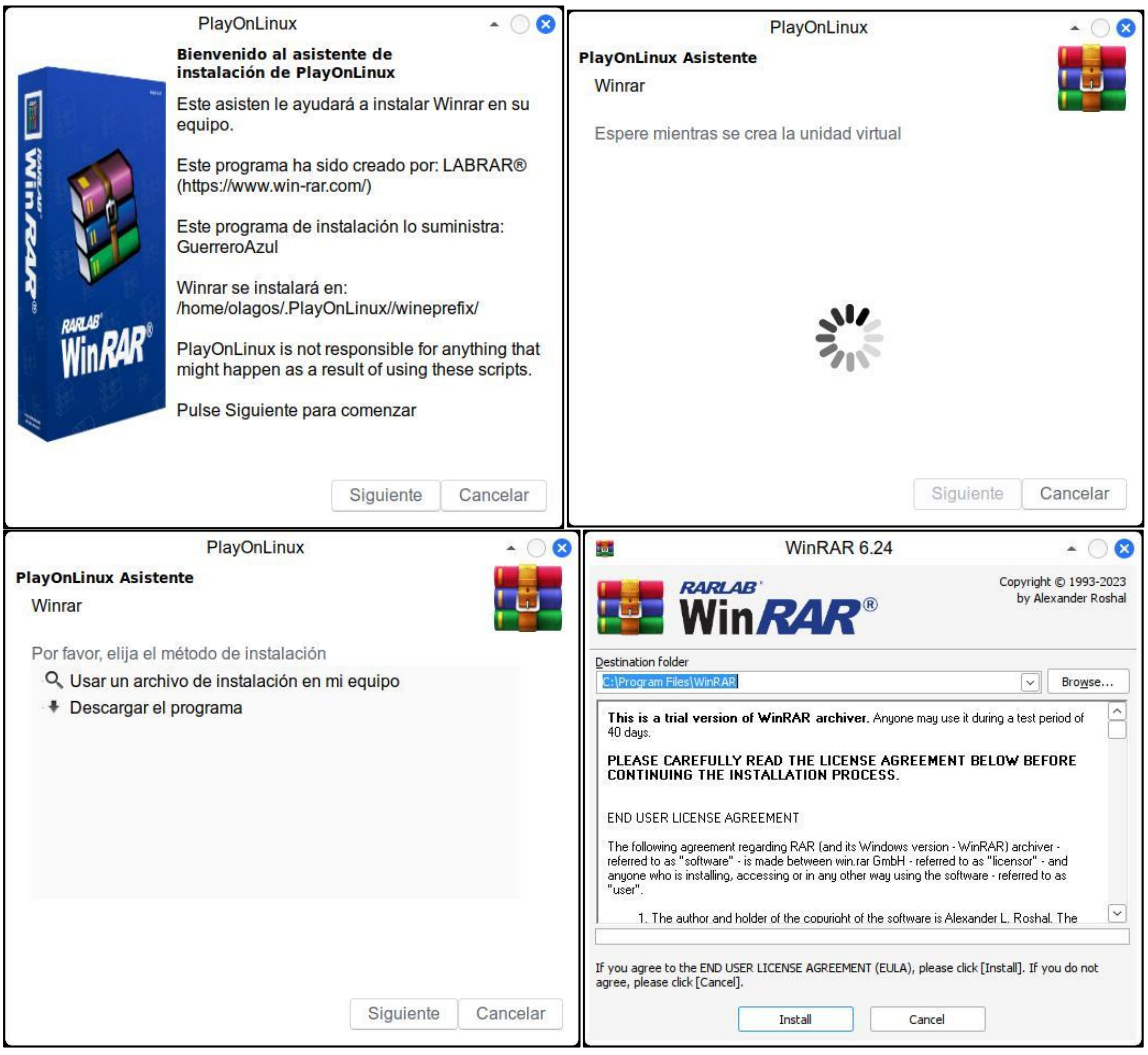
# Script start
POL_SetupWindow_InstallMethod "LOCAL,DOWNLOAD"
if [ "$INSTALL_METHOD" = "DOWNLOAD" ]; then
   POL_System_TmpCreate "$PREFIX"
   cd "$POL_System_TmpDir"
  POL_Download "$DOWNLOAD_URL" "$MD5_CHECKSUM"
  Setup="$POL_System_TmpDir/$FILE_INSTALL"
else
   cd "$HOME"
  POL_SetupWindow_browse "$(eval_gettext 'Please select the setup file to
  run.')" "$TITLE"
  Setup="$APP_ANSWER"
fi

# Install Program
POL_SetupWindows_wait "$(eval_gettext 'Please wait while $TITLE is
installed.')" "$TITLE"
POL_Wine "$Setup"

 # Shortcut
 POL_Shortcut "$PREFIX.exe" "$TITLE" "" "" "$CATEGORY"

# End script
POL_System_TmpDelete
```

```
POL_SetupWindow_Close
exit 0
```





10. Traduccion de script

Un capítulo dedicado a las traducciones de guiones; ¿es tan importante?

Sí. Los scripts de PlayOnLinux deben traducirse a todos los idiomas para que puedan utilizarlos el mayor número posible de personas.

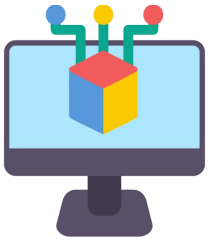
Para reducir la carga de trabajo de los traductores, los mensajes deben ser lo más idénticos posible entre los guiones (porque si el mensaje ya ha sido traducido, no será necesario traducirlo nuevamente para el nuevo guión). Por eso, hicimos una lista de los mensajes más comunes.

- Please select the setup file to run.
- Please insert the game media into your disk drive.
- Please wait while $TITLE is installed.
- $TITLE has been successfully installed.

Úsalos siempre que puedas. Si necesitas algún mensaje que no está en esta lista, sigue adelante y escríbelo en inglés. Lo que debes evitar son mensajes que no estén en esta lista pero que tengan el mismo significado que uno que sí lo esté.

El comando **`eval_gettext`** toma un mensaje en inglés como argumento y devuelve su traducción al idioma del usuario actual (según se determine automáticamente).

```
POL_SetupWindow_message "$(eval_gettext 'Please select the setup file to
run.')" "$TITLE"
```



11. Final del manual

Eso es todo, el tutorial termina aquí, debes tener los conocimientos suficientes para valerte por tu cuenta. Hay un anexo disponible en el próximo capítulo, que enumera todas las variables y todas las funciones que se pueden usar en los scripts de PlayOnLinux, porque hemos visto algo así como una cuarta parte de las funciones existentes. Uno de ellos puede resultar útil en un guión específico, así que considere echarle un vistazo a este anexo en ese momento. Sin embargo, tenga en cuenta que las funciones utilizadas en este tutorial serán todo lo que necesita para la mayoría de los scripts.

Para mas informacion visitar:

https://wiki.playonlinux.com/

Mas funciones en POL:

https://wiki.playonlinux.com/index.php/Scripting_-_Chapter_11:_List_of_Functions